

PROJECT DOCUMENTATION

CrimeGuard AI

AI-Powered Crime Resolution Prediction and Decision Support System

A Machine Learning Based Web Application for Predictive Crime Analytics

Submitted By

Aniket Ghantewad

Under the Guidance of

Rithik Raj Sir

Imarticus Learning

2026

Acknowledgement

I would like to express my sincere gratitude to my project guide for the valuable guidance, continuous support, and encouragement provided throughout the development of this project, CrimeGuard AI. Their insightful feedback and technical direction were instrumental in shaping the outcome of this work.

I am also thankful to the faculty members of my department for providing the necessary resources, infrastructure, and a conducive learning environment that made this project possible.

I extend my heartfelt thanks to my family and friends for their constant motivation and support during the course of this project. Finally, I acknowledge the open-source community and the creators of the tools and libraries used in this project, without which this work would not have been possible.

Aniket Ghanewad

Abstract

Law enforcement agencies across the world face the ongoing challenge of allocating limited investigative resources across a growing volume of reported crimes. CrimeGuard AI is a web-based, machine learning powered decision-support system designed to predict the probability that a reported crime case will be resolved, based on environmental, situational, and case-specific attributes such as crime type, severity, weapon involvement, lighting conditions, CCTV coverage, gang involvement, and suspect status.

The system is built on a Logistic Regression classification model trained on a structured crime dataset comprising ten thousand records and thirty-two attributes spanning fifteen countries. The trained model is deployed through a Flask-based backend that exposes a prediction API, while an interactive front-end allows an investigator to enter case details and instantly receive a resolution probability, a risk classification, a recommended number of officers to deploy, a set of prioritised action recommendations, and a breakdown of the factors that most influenced the prediction.

In addition to real-time prediction, the application maintains a locally stored case history so that previously analysed cases can be reviewed, revisited, or exported as a case ticket. This document presents the complete lifecycle of the project — from the problem definition and dataset analysis, through model development and evaluation, to the design and implementation of the final web application — along with a discussion of results and directions for future enhancement.

Table of Contents

1. Introduction	5
2. Literature Review	6
3. System Requirements.....	7
4. Dataset Description	8
5. System Design	10
6. Model Development.....	11
7. Model Evaluation	12
8. System Implementation	13
9. Website Modules	15
10. Results & Discussion	19
11. Conclusion.....	20
12. Future Scope	21
13. References	22

1. Introduction

1.1 Background

Crime resolution — the successful investigation and closure of a reported criminal case — depends on a complex mix of environmental, procedural, and situational factors. Traditional crime analysis has largely relied on historical statistics and manual case review, which makes it difficult for investigators to quickly judge which cases are likely to be resolved and which require immediate additional attention. With the growth of digitised crime records, machine learning offers an opportunity to convert this historical data into predictive insight that can directly support day-to-day policing decisions.

1.2 Problem Statement

In an era of rising urban challenges and limited law enforcement resources, the critical question is:

How can we predict the likelihood of crime resolution and support smarter, faster investigative decisions using data-driven analysis?

Crime incidents vary significantly depending on multiple environmental and case-specific factors such as crime type and severity, weapon involvement, suspect status, CCTV coverage, lighting conditions, gang involvement, and the geographic and area context in which the crime occurs. Traditional crime analysis methods depend mainly upon historical reports and descriptive statistics, making it difficult for investigators to quickly assess which cases are likely to be resolved and which require immediate additional attention. This gap creates a need for a predictive, data-driven system that can convert case-level information into an actionable resolution probability, risk classification, and recommended investigative response — enabling law enforcement agencies to allocate limited resources more effectively.

1.3 Objectives

- To analyse a real-world structured crime dataset and identify the factors most associated with case resolution.
- To build and compare multiple machine learning classification models capable of predicting whether a case will be resolved.
- To select and tune the best-performing model and deploy it behind a prediction API.
- To design and develop a web application that allows an investigator to submit case details and instantly receive a prediction, risk level, officer recommendation, and actionable guidance.
- To provide a case history module so that generated predictions can be stored, reviewed, and exported.

1.4 Scope of the Project

The project covers the complete pipeline of a predictive analytics application: dataset cleaning and exploration, feature engineering, model training and evaluation, and the design and implementation of a Flask-based web application with three functional pages (Home, History, About) and a detailed Case Report view. The prediction is based solely on the nine case attributes captured through the web form; it does not integrate with any live police records management system, and the case history is stored locally in the user's browser rather than in a central database.

2. Literature Review

Predictive analytics in criminology has been an active area of research and practice for over two decades, ranging from hotspot mapping techniques that forecast where crimes are likely to occur, to case-level models that estimate the probability of clearance or resolution. Classical approaches relied heavily on descriptive statistics and rule-based scoring systems derived from historical case-closure rates, which offered transparency but struggled to capture non-linear interactions between multiple contributing factors.

The introduction of supervised machine learning classifiers — including logistic regression, decision trees, and ensemble methods such as random forests and gradient boosting — has enabled more accurate modelling of case outcomes by learning patterns directly from structured incident data. Logistic regression in particular remains widely used in criminal justice analytics due to its interpretability: model coefficients can be directly related to the contribution of each factor, which is valuable in a domain where decisions must be explainable to investigators and, ultimately, to the public.

Existing predictive policing tools have also been criticised in the literature for risks such as reinforcing historical bias present in training data and providing recommendations without sufficient human oversight. This motivates the design choice, followed in this project, of treating the system strictly as a decision-support aid — surfacing probability, risk, and contributing factors — rather than an autonomous decision-maker, with a clear disclaimer that outputs must be verified against evidence and professional judgement before any operational action is taken.

3. System Requirements

3.1 Hardware Requirements

- Processor: Intel Core i3 / AMD equivalent or higher
- RAM: 4 GB minimum (8 GB recommended for model training)
- Storage: 500 MB free disk space
- Display: Standard monitor with a modern web browser

3.2 Software Requirements

- Operating System: Windows / Linux / macOS
- Programming Language: Python 3
- Backend Framework: Flask
- Machine Learning Libraries: scikit-learn, pandas, numpy, joblib
- Visualisation Libraries (analysis phase): Matplotlib, Seaborn, Plotly
- Frontend: HTML5, CSS3, JavaScript (vanilla)
- Development Tools: Jupyter Notebook, Visual Studio Code
- Browser: Any modern browser (Chrome, Edge, Firefox) supporting HTML5 and localStorage

3.3 Technology Stack Summary

Layer	Technology Used
Data Analysis & Model Training	Python, Pandas, NumPy, Scikit-learn, Jupyter Notebook
Model Persistence	Joblib (.pkl model package)
Backend / API	Flask (Python)
Frontend	HTML5, CSS3, JavaScript
Data Storage (Case History)	Browser localStorage (client-side, no external database)
Model Deployment	Flask development server (app.run)

4. Dataset Description

The model is trained on the Crime Rate and Safety Analysis dataset, a structured CSV file containing 10,000 incident records described by 32 attributes, covering reported crimes from the year 2021 to 2025 across 15 countries. Each record represents a single crime incident along with contextual, investigative, and outcome-related information.

4.1 Key Attributes

Attribute	Description
incident_id	Unique identifier for each crime record
year / month / day_of_week / season / time_of_day	Temporal attributes of the incident
country / area_type	Location of the incident (15 countries; Urban, Suburban, Rural, Remote)
crime_type	Category of crime (Theft, Assault, Robbery, Kidnapping, etc. — 15 types)
crime_severity_score	Numeric severity rating of the incident (1–10)
weapon_used	Weapon involved, if any (Firearm, Knife, Blunt Object, Vehicle, Chemical, Unknown)
cctv_coverage / lighting_condition	Environmental / evidentiary conditions at the scene
gang_related / drug_related / repeat_offender	Contextual flags about the incident
suspect_status	Current status of the suspect (Convicted, Arrested, At Large, etc.)
response_time_minutes / officers_assigned / investigation_duration_days	Operational response metrics
victim_count / victim_gender / victim_age_group / injuries_reported / fatalities	Victim profile information
financial_loss_usd / safety_index	Impact and area-safety indicators
crime_resolved	Target variable — whether the case was resolved (Yes / No)

4.2 Data Preprocessing

The dataset was first checked for duplicate records and missing values. Missing entries in the `weapon_used` column were imputed with the category "Unknown", while the small number of remaining rows with missing values in other columns were dropped. The target column `crime_resolved` was mapped from its Yes/No text values to a binary 1/0 label for model training.

4.3 Feature Selection

For the prediction model, nine attributes that are realistically available to an investigator at the time a case is opened were selected as model inputs: country, area_type, crime_type, crime_severity_score, weapon_used, cctv_coverage, gang_related, lighting_condition, and suspect_status. Categorical features were converted into numeric form using one-hot encoding before being passed to the model.

5. System Design

5.1 Architecture Overview

CrimeGuard AI follows a simple client–server architecture. The browser-based frontend collects case details through a structured form and sends them to the Flask backend as a JSON request. The backend loads the pre-trained model package (model and its expected input columns) at start-up, encodes the incoming case in the same format used during training, and returns a structured JSON response containing the prediction, probability, risk level, officer recommendation, prioritised actions, and factor contributions. The frontend then renders this response into the interactive Case Report view and stores a copy of it in the browser's local storage for the History module.

5.2 Data Flow

- 1. User fills in case details on the Home page (location, environment/evidence, suspect profile).
- 2. Frontend JavaScript sends the form data to the `/api/predict` endpoint.
- 3. Flask backend builds a single-row DataFrame from the form, one-hot encodes it, and aligns it to the model's expected column order.
- 4. The trained Logistic Regression model returns a class prediction and a resolution probability.
- 5. Backend derives risk level, officer count, recommended actions, and per-factor signal contributions from the case values and model coefficients.
- 6. The JSON result is returned to the browser, rendered as a Case Report, and saved to `localStorage` for the Case History page.

5.3 Application Routes

Route	Method	Purpose
<code>/</code>	GET	Renders the Home page with the case-input form
<code>/history</code>	GET	Renders the Case History (ledger) page
<code>/about</code>	GET	Renders the About page
<code>/case-report</code>	GET	Renders the detailed Case Report page
<code>/api/predict</code>	POST	Accepts case details (JSON/form) and returns the prediction as JSON
<code>/predict</code>	POST	Legacy server-rendered prediction route, kept for compatibility

6. Model Development

6.1 Feature–Target Separation

The nine selected input features (country, area_type, crime_type, crime_severity_score, weapon_used, cctv_coverage, gang_related, lighting_condition, suspect_status) were separated from the target variable crime_resolved. Categorical features were one-hot encoded using pandas.get_dummies with drop_first=True to avoid redundant columns, and the resulting feature matrix was split into training and testing sets using an 80/20 stratified split (random_state = 42) to preserve the original class ratio.

6.2 Candidate Models

Three candidate classification algorithms were trained and compared on the held-out test set: Logistic Regression, Decision Tree, and Random Forest.

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	0.7614	0.7486	0.7614	0.7493	0.8010
Random Forest	0.7620	0.7500	0.7620	0.7365	0.7895
Decision Tree	0.7310	0.7201	0.7310	0.7238	0.7458

Although Random Forest achieved a marginally higher raw accuracy, Logistic Regression produced the best F1 score and ROC-AUC, and — critically — offers directly interpretable coefficients that the application uses to explain each prediction to the investigator (the "Prediction Factors" breakdown). Logistic Regression was therefore selected as the model to take forward for tuning and deployment.

6.3 Hyperparameter Tuning

A grid search with 5-fold cross-validation (GridSearchCV) was performed over the regularisation strength C ({0.01, 0.1, 1, 10, 100}), the solver ({liblinear, lbfgs}), and class-weighting ({None, balanced}), optimising for accuracy. The search identified C = 100, solver = lbfgs, and class_weight = None as the best configuration, with a cross-validated accuracy of approximately 0.760.

6.4 Model Persistence

The final tuned Logistic Regression model, together with the ordered list of feature columns produced during one-hot encoding, was serialised into a single package using joblib and saved as crime_resolution_model.pkl. This package is loaded once when the Flask application starts, ensuring that every incoming case is encoded against the exact same column structure used during training.

7. Model Evaluation

7.1 Confusion Matrix

The confusion matrix for the final model on the test set (1,580 cases) is as follows:

	Predicted: Not Resolved	Predicted: Resolved
Actual: Not Resolved	984	125
Actual: Resolved	252	219

7.2 Classification Report

Class	Precision	Recall	F1-Score	Support
Not Resolved (0)	0.80	0.89	0.84	1109
Resolved (1)	0.64	0.46	0.54	471
Accuracy			0.76	1580
Macro Avg	0.72	0.68	0.69	1580
Weighted Avg	0.75	0.76	0.75	1580

7.3 Discussion

The final model correctly classifies unresolved cases with high recall (0.89), reflecting the fact that unresolved cases make up the majority class in the dataset. Performance on the minority "resolved" class is comparatively lower (recall 0.46), which is a common characteristic of case-outcome data where resolution depends on many factors outside the nine attributes captured by the model, such as unrecorded investigative effort. The overall ROC-AUC of approximately 0.80 indicates that the model has a good ability to rank cases by resolution likelihood, which supports its intended use as a triage and prioritisation aid rather than a certainty-guaranteed predictor.

8. System Implementation

8.1 Project Structure

The deployed application is organised into the following folder structure:

Path	Contents
app.py	Flask application: routes, prediction logic, and helper functions
crime_resolution_model.pkl	Serialised model package (trained model + expected columns)
crime_rate_and_safety_analysis_dataset.csv	Source dataset used for training and analysis
Crime Rate And Safety Analysis.ipynb	Jupyter notebook covering EDA, preprocessing, and model training
templates/	Jinja2 HTML templates: base.html, index.html, history.html, about.html, case_report.html
static/css/style.css	Application styling (dark, dashboard-style theme)
static/js/	Client-side logic: main.js, predict-form.js, history.js, case-report.js

8.2 Backend Logic

The Flask backend (app.py) loads the model package at start-up and exposes helper functions that transform a raw prediction into a complete decision-support response:

- `build_input_df()` — assembles the submitted form values into a single-row DataFrame.
- `encode()` — one-hot encodes the input and aligns it to the model's expected column order.
- `risk_level()` — converts the resolution probability into a Low / Moderate / High / Critical risk label.
- `officer_deployment()` — recommends a number of officers based on the crime severity score.
- `recommend_actions()` — generates a prioritised list of recommended actions (e.g. deploy armed response, increase CCTV surveillance, notify anti-gang unit) based on case attributes and the predicted probability.
- `signal_contributions()` — computes each input field's contribution to the model's log-odds output using the case's own encoded values and the model's learned coefficients, powering the "Prediction Factors" chart.
- `signal_profile()` — produces a 0–1 favourability score for key environmental factors (CCTV, lighting, suspect clarity, gang involvement, severity), used for supporting visual summaries.

8.3 Frontend Implementation

The frontend is built with plain HTML, CSS, and JavaScript rendered through Flask/Jinja2 templates, sharing a common `base.html` layout with a persistent navigation bar (Home, History, About) and a model status indicator. `predict-form.js` handles form submission and calls the `/api/predict` endpoint; `case-report.js` renders the detailed prediction output, charts, and PDF export on the Case Report page; and `history.js` manages saving, listing, viewing, and deleting past predictions in the browser's `localStorage`.

9. Website Modules

9.1 Home Module

The Home page is the entry point of the application. It presents the case-input form, organised into three logical sections: Case Location (country, area type, crime type, severity score), Environment & Evidence (weapon used, CCTV coverage, lighting condition, gang involvement), and Suspect Profile (suspect status). Once the details are filled in, the investigator clicks Analyze Case to submit the form to the prediction API.

The screenshot displays the 'CrimeGuard AI' web application interface. At the top, a navigation bar includes 'HOME', 'HISTORY', and 'ABOUT' links, along with a 'MODEL ONLINE' status indicator. Below the navigation bar, three tabs are visible: 'MODEL ONLINE', 'REAL CASE PATTERNS', and 'LIVE PREDICTION'. The main heading reads 'Crime Resolution, Predicted. Actioned.' followed by a subtext: 'Transform case details into actionable insights with CrimeGuard AI. Get resolution probability, risk assessment, officer deployment recommendations, and prioritised action plans — in under a second.'

The form is organized into three sections:

- Case Location** (Where and what kind of incident):
 - COUNTRY**: A dropdown menu with 'Select country'.
 - AREA TYPE**: A dropdown menu with 'Select area'.
 - CRIME TYPE**: A dropdown menu with 'Select crime'.
 - SEVERITY SCORE (1 - 50)**: A text input field with the value '5'.
- Environment & Evidence** (Surveillance, lighting, and weapon used):
 - WEAPON USED**: A dropdown menu with 'Select weapon'.
 - CCTV COVERAGE**: A dropdown menu with 'Select level'.
 - LIGHTING CONDITION**: A dropdown menu with 'Select lighting'.
 - GANG RELATED**: A dropdown menu with 'Select'.
- Suspect Profile** (Current status of any identified suspect):
 - SUSPECT STATUS**: A dropdown menu with 'Select status'.

At the bottom of the form is a large blue button labeled 'ANALYZE CASE'. The footer contains the text: '© 2024 CRIMEGUARD AI - AI-POWERED CRIME RESOLUTION PREDICTION - SECURE'.

Figure 9.1 — Home page: case-input form for crime resolution prediction

9.2 History Module

The History page displays a ledger of every prediction generated on the current device. Each entry shows the case ID, crime type, location, severity, and timestamp, along with colour-coded risk indicators, and provides View and Delete actions. As stated on the page itself, all case history is stored locally in the browser and never leaves the user's device.

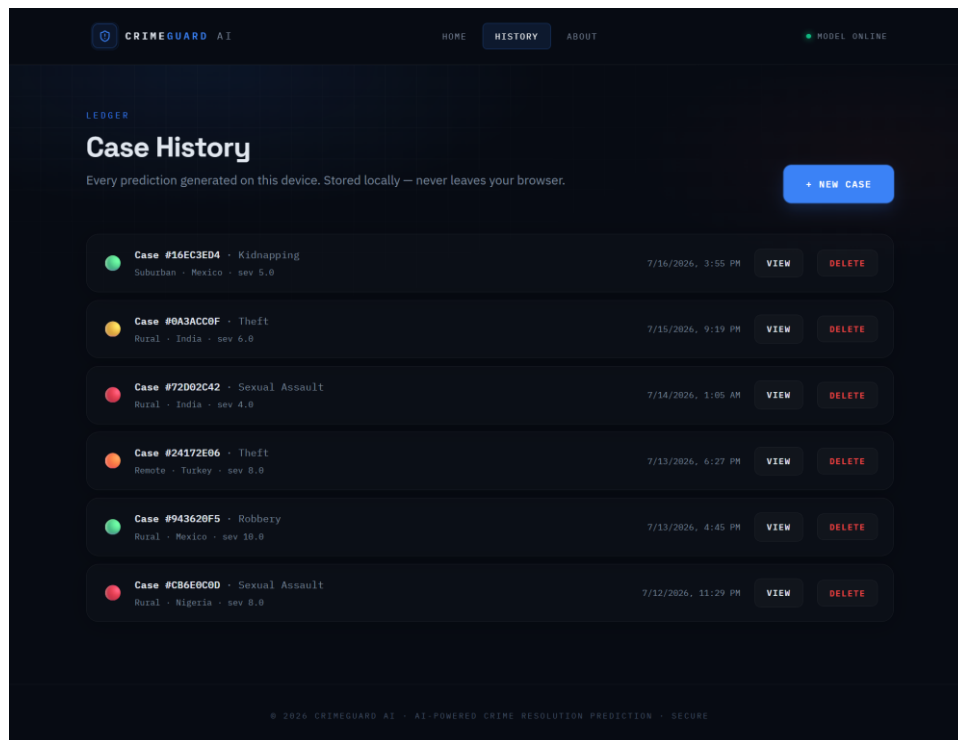


Figure 9.2 — Case History page listing previously analysed cases

9.3 Case Report Module

The Case Report page presents the full output of a single prediction: the overall verdict (e.g. "Likely to Resolve"), a risk badge and severity score, a circular gauge showing the resolution probability, and quick-reference cards for officers recommended, suspect status, weapon, and location. Below this, a Prediction Factors chart visualises the relative influence of each input field on the outcome, a Recommended Actions panel lists prioritised next steps, and a Prediction Log shows a step-by-step trace of how the prediction was produced. The page also allows the investigator to download the result as a Case Ticket PDF.

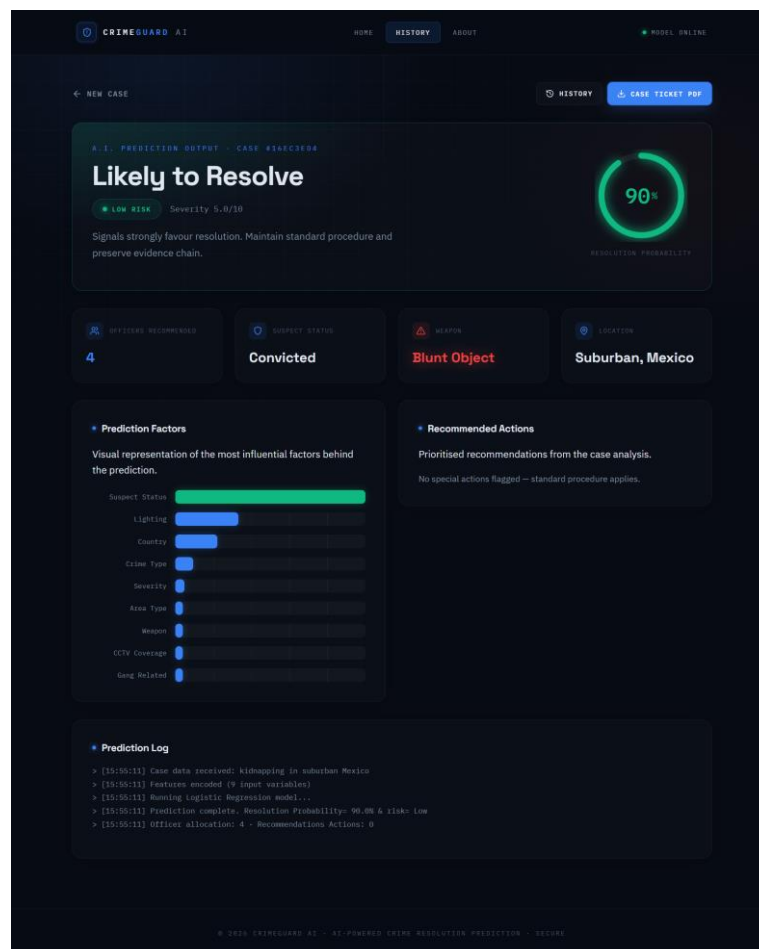


Figure 9.3 — Case Report page showing prediction output for a sample case

9.4 About Module

The About page explains the purpose of CrimeGuard AI, why data-driven prioritisation matters for resource-constrained police departments, and how the three-step process (Enter Details, AI Analyses, Get Results) works. It also lists the benefits of the system and includes a clear disclaimer stating that the tool supports investigative decision-making but that final operational decisions must always rely on verified evidence, official protocols, and professional judgement.

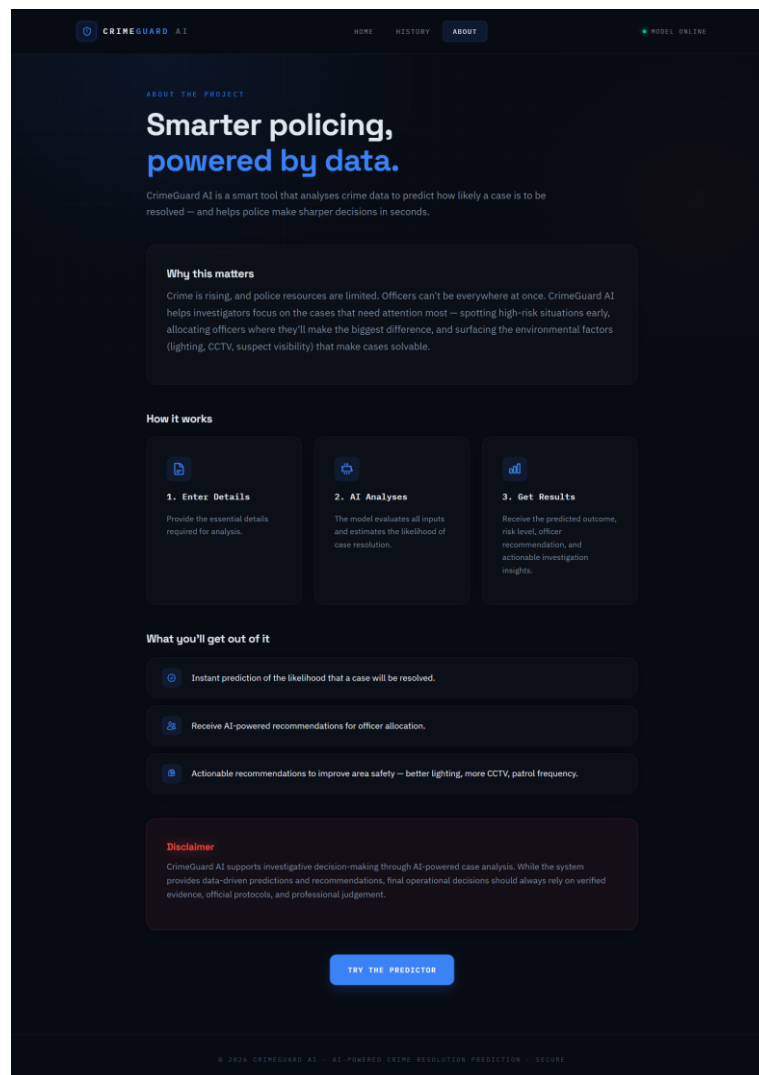


Figure 9.4 — About page describing the project and its intended use

10. Results & Discussion

To illustrate the end-to-end behaviour of the system, consider a sample case submitted through the Home page: a kidnapping incident in a suburban area of Mexico, with a severity score of 5.0, a blunt object involved, and a suspect already convicted. The system returns a resolution probability of 90%, classifies the case as Low Risk, and recommends deploying 4 officers. Because the probability is high and no high-severity or high-risk flags are triggered, the Recommended Actions panel reports that no special actions are required beyond standard procedure.

The Prediction Factors breakdown for this case shows Suspect Status as, by a wide margin, the most influential factor pushing the prediction toward resolution — consistent with the intuitive expectation that a case with a convicted suspect is overwhelmingly likely to be recorded as resolved. Lighting Condition and Country contribute smaller, secondary effects, while Crime Type, Severity, Area Type, Weapon, CCTV Coverage, and Gang Related involvement contribute comparatively minor adjustments for this particular case.

This pattern is consistent with the coefficients learned during model training and with the model evaluation results discussed in Section 7: the model is most confident when the suspect's current status strongly indicates an outcome (Convicted or Arrested versus At Large or Unknown), while environmental factors act as secondary modifiers of that base signal. Overall, the deployed application successfully reproduces the behaviour observed during offline evaluation, and the Case History module confirms that repeated predictions for different crime types, countries, and severities produce risk levels and officer recommendations that vary sensibly with the input case profile.

11. Conclusion

This project has demonstrated a complete pipeline for predictive crime-resolution analytics, from dataset exploration and preprocessing through model comparison, tuning, and deployment, to a fully functional web application. The final Logistic Regression model achieves an accuracy of approximately 76% and a ROC-AUC of approximately 0.80 on held-out data, and its interpretable coefficients allow the CrimeGuard AI application to not only predict an outcome but also explain the leading factors behind that prediction and recommend concrete next steps, such as officer deployment levels and priority actions.

The resulting system, CrimeGuard AI, packages this model behind an intuitive, dashboard-style web interface with dedicated Home, History, Case Report, and About modules, giving investigators a fast, consistent, and transparent way to triage incoming cases. As emphasised throughout the application itself, the tool is intended strictly to support — not replace — human investigative judgement.

12. Future Scope

- Integrating the application with a persistent, server-side database (e.g. PostgreSQL or MongoDB) so that case history can be shared across devices and users rather than being limited to browser localStorage.
- Evaluating additional and ensemble models, such as Gradient Boosting and XGBoost, and applying advanced imbalance-handling techniques to improve recall on the minority "resolved" class.
- Adding user authentication and role-based access control for different levels of law-enforcement personnel.
- Extending the system to ingest live or near-real-time crime reports rather than requiring manual form entry.
- Adding geospatial mapping and hotspot visualisation to complement the case-level predictions.
- Supporting multiple languages and a dedicated mobile application for field use.

13. References

1. Crime Rate and Safety Analysis Dataset (`crime_rate_and_safety_analysis_dataset.csv`), project dataset used for model training and exploratory analysis.
2. Flask Documentation, Pallets Projects (flask.palletsprojects.com).
3. Pandas Documentation (pandas.pydata.org).
4. Plotly, Matplotlib, and Seaborn documentation, used for exploratory data visualisation during the analysis phase.
5. Course/departmental reference material on machine learning classification techniques and predictive analytics in criminology.